

Enterprise AI Copilot for ERP

Full Build Plan — Problem, Solution, Architecture, and Stage-by-Stage Execution (Project 1)

Update: This revision swaps the LLM inference layer from local-only Ollama to a provider-agnostic setup that defaults to OpenRouter (hosted API, free-tier models available), with local Ollama kept as an optional fallback. Rationale is explained in Section 4a. All other sections are unchanged from the original plan.

1. The Problem This Project Solves

In a real ERP system (SAP S/4HANA and similar), day-to-day work like finding an invoice, checking whether an approval is stuck, or spotting a suspicious payment requires knowing exactly which transaction codes, reports, or menu paths to use. New employees are slow because the interface is built around structured forms, not questions. An AP (Accounts Payable) clerk who wants to know “which invoices from Vendor X are still unpaid” has to know how to build that query in the ERP UI. This project builds a natural-language layer on top of ERP-style data so a user can just ask the question in plain English and get a grounded, accurate answer — the same problem SAP, BMW, and Siemens are actively solving with AI copilots bolted onto their ERP and asset systems.

2. What the Finished System Actually Does

A user logs in as one of two demo roles (AP Clerk or Manager) and lands on a chat-style interface. They can type a free-text question, or click a suggested prompt. Behind the scenes, three things can happen for any given question:

- **Structured lookup** — the question maps cleanly to a known ERP entity (“show invoices from Vendor X last month”) so the copilot calls a direct data endpoint and returns real rows.
- **Policy/knowledge question** — the question is about a rule, not a record (“what's the approval limit for a manager?”) so the copilot retrieves the relevant policy text via RAG and answers from it.
- **Hybrid question** — the question needs both (“is this invoice over the approval threshold?”) so the copilot retrieves the policy AND queries the live invoice data, then combines the two in its answer.

If the copilot can't find grounding for a question in either the data or the policy documents, it says so explicitly (“I don't have that information”) rather than guessing — this single guardrail is what separates a toy chatbot from something that looks trustworthy enough for enterprise use.

3. System Architecture — How the Pieces Fit

Request flow for a typical question, end to end:

1. React frontend sends the logged-in user's question + JWT cookie to POST /copilot/query on FastAPI.
2. FastAPI validates the JWT, extracts the user's role (clerk vs manager), and passes the question to the RAG layer.
3. The question is embedded (sentence-transformers) and matched against a Chroma vector index of your ERP policy documents.
4. The top-matching policy chunks are inserted into a prompt template, along with the user's role and question.

5. If the question needs live data, the copilot logic also calls your own internal endpoints (/invoices, /approvals/pending) and folds the returned rows into the same prompt as context.
6. The assembled prompt goes to the configured LLM provider — by default a hosted model via OpenRouter, or a local Ollama model if configured — which generates the final natural-language answer.
7. FastAPI returns the answer (plus which sources/records were used) to React, which renders it in the chat window.

The key architectural idea to be able to explain in an interview: retrieval (RAG) supplies unstructured knowledge (policies, rules), while your own CRUD endpoints supply structured facts (actual invoice rows). The LLM's only job is language — turning both into a coherent answer — never inventing numbers itself. Which LLM provider generates that language (hosted API vs. local model) is an interchangeable implementation detail, not a core architectural decision.

4. Exactly How Each Piece of the Stack Is Used

FastAPI (backend)

- Hosts all REST endpoints: auth, mock ERP data (invoices, approvals), the copilot query endpoint, and the doc-indexing endpoints.
- Pydantic models validate every request body (e.g. a query must include a non-empty question string) and every response shape, so the frontend always gets predictable JSON.
- Dependency injection supplies the current authenticated user + role to any endpoint that needs it, keeping auth logic in one place.

LLM Inference — OpenRouter (default) with Ollama fallback

- FastAPI calls a small internal adapter (e.g. `get_llm_response(prompt)`) that hits the OpenRouter chat-completions API (<https://openrouter.ai/api/v1/chat/completions>) using a free-tier model. Which exact model is free changes over time, so the model name is read from an environment variable, not hard-coded.
- The same adapter can point at a local Ollama instance instead (llama3.2:3b or similar) by swapping one environment variable — this keeps the system provider-agnostic and demonstrates that the architecture doesn't depend on any single vendor.
- A fixed system prompt defines the copilot's persona and hard rule: “only answer from the provided context; if the context doesn't contain the answer, say you don't know.” This rule lives in the prompt template, not in provider-specific code, so it holds regardless of which backend is generating the answer.
- API key for OpenRouter is stored in a `.env` file (never committed) and injected via FastAPI's config/settings object.

sentence-transformers + Chroma (RAG)

- sentence-transformers turns both your policy documents and the user's question into vectors (numeric representations of meaning) using the same free local model.
- Chroma stores those vectors locally and, given a question's vector, returns the most similar policy chunks — this is the “retrieval” in retrieval-augmented generation.
- Metadata on each chunk (e.g. `company_code`, `doc_type`) lets you filter retrieval, mirroring how a real ERP separates data by entity/fiscal year.

React (frontend)

- A login screen posts credentials to /auth/login and stores the returned JWT as an HttpOnly cookie (handled server-side, not by JS, for basic security).
- The chat screen is a single component with local state for message history, a text input, and 3-4 suggested-prompt buttons that pre-fill common questions.
- Loading and error states are shown while waiting on /copilot/query so the UI never looks frozen or silently fails.

JWT auth (hand-rolled)

- Two hardcoded demo users (one per role) are enough — the point is to demonstrate role-aware behavior, not build a full user-management system.
- The JWT payload carries user_id and role; FastAPI reads the role on every request to decide what the copilot is allowed to answer (e.g. only managers can ask about approval overrides).

Docker / Docker Compose

- One service for FastAPI, one for React (or serve the React build as static files from FastAPI to simplify further). Ollama is now an optional third service, only included if OLLAMA mode is selected.
- docker-compose up brings the whole system online with no manual setup steps — this is what makes the project reproducible for anyone reviewing it. With OpenRouter as the default, this also means reviewers with no GPU/RAM to spare can run the full demo without pulling any model weights locally.

4a. Why OpenRouter Instead of Local-Only Ollama

The original plan specified a locally-run model (llama3.1:8b or mistral:7b) as the zero-cost LLM layer. On constrained hardware (e.g. 8GB RAM laptops), a 7–8B model competes with the OS, Chroma, FastAPI, and the React dev server for the same memory, which leads to slow responses, swapping, and occasional instability — risks best avoided right before a demo recording.

OpenRouter is used instead as the default inference path: it exposes a single unified API to many hosted models, including models on a free tier, so the €0-cost constraint is preserved without requiring local hardware capable of running a large model well. This also mirrors how real ERP copilots (SAP Joule, Copilot for Dynamics, etc.) are actually built — against a hosted inference endpoint, not a model running on the developer's own laptop.

Local Ollama is retained as a supported fallback mode (not removed from the plan) so the project can still demonstrate that the architecture is provider-agnostic: swapping between a hosted API and a local model is a one-line configuration change, not a code rewrite. This distinction — a clean adapter boundary between “prompt assembly” and “whichever model answers it” — is worth calling out explicitly in an interview, since it is exactly the kind of design decision that separates a production-minded system from a hard-coded demo.

Trade-off to be honest about: OpenRouter requires an internet connection and an account (API key), so the project is no longer fully offline. If a fully offline, zero-external-dependency demo is a hard requirement, use the Ollama fallback mode instead for the recorded demo.

5. Data Model

Your synthetic ERP dataset (CSV/JSON, loaded into SQLite or an in-memory store) needs three tables:

Table	Key fields	Purpose
vendors	vendor_id, name, company_code	Who invoices come from
invoices	invoice_id, vendor_id, amount, date, status	The core object the copilot queries and reasons about
approvals	approval_id, invoice_id, approver_role, status	Defines the “approval limit” and “pending approvals” scenarios

Your policy documents (plain text/Markdown files you write yourself) should include things like: “Invoices under €1,000 are auto-approved. Invoices €1,000–€10,000 require Manager approval. Invoices above €10,000 require two Manager approvals.” These are what the RAG layer retrieves for policy questions, and what lets you test the hybrid case (“is invoice #1042 over the threshold?”).

6. Core API Contracts

GET /invoices?vendor=&start_date=&end_date=&status=

Returns a filtered list of invoice rows from the mock ERP dataset. Powers the “find invoice” workflow directly, and is called internally by the copilot for hybrid questions.

GET /approvals/pending?role=

Returns approvals awaiting the current user’s role. Powers the “summarize open approvals” workflow.

POST /copilot/query

Body: { “question”: string }. Internally: retrieves policy chunks, optionally calls /invoices or /approvals/pending, assembles the prompt, calls the configured LLM provider (OpenRouter by default, Ollama if configured), returns { “answer”: string, “sources”: [...] }. The sources field (which chunks/records were used) is what lets you show traceability in the UI — a detail enterprise reviewers specifically look for.

POST /auth/login

Body: { “username”: string, “password”: string }. Returns a JWT set as an HttpOnly cookie on success.

7. Detailed Build Stages

Stage 0 — Scope, data, and policy docs (2–3 days)

- Pick exactly 3 workflows: find invoice by vendor/date; summarize open approvals for a manager; flag invoices above the approval threshold.
- Write the vendors/invoices/approvals CSVs (50–100 rows total is enough to feel real).
- Write 3–5 short policy documents covering approval thresholds and basic invoice rules — these are what RAG will retrieve from.
- Deliverable: a one-page design doc in the repo listing the 3 workflows, the data schema, and 2–3 example user questions per workflow.

Stage 1 — FastAPI + mock ERP + auth (no LLM yet)

- Implement GET /invoices, GET /approvals/pending, POST /auth/login with the two hardcoded demo users and JWT-in-cookie auth.

- Deliverable: a working, testable API you can hit with curl or Postman that returns real filtered data and enforces login — commit and tag this as v0.1.

Stage 2 — RAG copilot backend

- Load policy docs into Chroma via sentence-transformers embeddings; write the index-docs step as a one-time setup script.
- Build the LLM adapter first: a single function that takes a prompt and returns text, backed by OpenRouter by default (with an Ollama code path behind the same interface). Get a free OpenRouter API key and test the adapter in isolation with a curl/script call before wiring it into /copilot/query.
- Implement POST /copilot/query: embed question -> retrieve top-k chunks -> decide if live data is needed (simple keyword check on the question is fine — e.g. contains “invoice #” or “vendor”) -> assemble prompt -> call the LLM adapter -> return answer + sources.
- Hard-code the guardrail into the system prompt: never answer from outside the provided context.
- Deliverable: you can POST a question and get a grounded answer, with a visible list of which policy chunks or invoice rows were used.

Stage 3 — React chat UI

- Login screen, chat window with message history in local state, 3-4 suggested-prompt buttons matching your 3 workflows, loading/error states.
- Deliverable: a click-through demo — log in as AP Clerk, ask a data question; log in as Manager, ask a policy question — both work end to end in the browser.

Stage 4 — Docker, docs, and a recorded demo

- docker-compose.yml bringing up FastAPI + React (served as static files, or its own container). Ollama is an optional third service, only started if LLM_PROVIDER=ollama is set; default docker-compose.yml assumes OpenRouter and requires only an API key, no model download.
- README with setup instructions, an architecture diagram (even a simple boxes-and-arrows sketch is fine), how to switch providers (OpenRouter vs local Ollama), and 3 example questions to try.
- Record a 2–3 minute screen capture of the full demo (login -> data question -> policy question -> hybrid question) and link it in the README, so the project is provably working even without anyone running it themselves.

8. Zero-/Low-Cost Stack Recap

Component	Choice	Cost
LLM (default)	OpenRouter, free-tier hosted model (provider/model set via env var)	€0
LLM (fallback)	Ollama (llama3.2:3b recommended for 8GB RAM machines), local	€0
Embeddings	sentence-transformers, local	€0
Vector DB	Chroma, embedded/local	€0
Relational storage	SQLite (or in-memory)	€0

Auth	Hand-rolled JWT	€0
Hosting	Local Docker Compose + recorded demo video	€0

Note: OpenRouter free-tier model availability changes over time — verify current free models at build time and set the model name via environment variable rather than hard-coding it.

9. Positioning for SAP / BMW / Siemens-Style Roles

“Enterprise AI Copilot for ERP (mock SAP S/4HANA-style workflows) — a full-stack system combining a hosted LLM via a provider-agnostic adapter (OpenRouter by default, with a local Ollama fallback), retrieval-augmented generation over policy documents, live structured-data lookups, and role-based access control, built and demonstrated at zero infrastructure cost.” Be ready to explain the RAG-vs-structured-data split above out loud, and to explain why the LLM layer is abstracted behind an adapter — that distinction is exactly what interviewers probe for when they want to know if you actually understand RAG and system design, or just called an API.